

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	Yaswanthika Shree S	Reg. No.:	192565069
Section / Batch:	B.E -CSE-Cyber Security	Date:	22.07.2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q71. Application: Smart Classroom Attendance System

A smart attendance system checks each student ID exactly once while marking presence. Identify the concepts of time complexity and space complexity, explain how the number of operations and memory usage grow with input size, and determine the asymptotic complexities of this process.

Q72. Application: University Result Comparison System

A result analysis system compares each student's score with every other student's score to prepare rank statistics. Recall what pairwise comparison implies in complexity analysis, explain why execution time increases rapidly, and compute both the time complexity and space complexity using asymptotic notation

Code of Conduct

I Yaswanthika Shree S(192565069) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yaswanthika Shree S

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Application-Based Question

Q71. Application: Smart Classroom Attendance System

Problem: A smart attendance system checks each student ID exactly once while marking presence. Identify the concepts of time complexity and space complexity, explain how the number of operations and memory usage grow with input size, and determine the asymptotic complexities of this process.

1. Concept of Time Complexity

Time complexity measures how the number of basic operations performed by an algorithm grows as the size of the input (n) increases. Here, the input is the list of ' n ' student IDs, and the system performs one checking/marking operation per student ID.

2. Concept of Space Complexity

Space complexity measures the amount of extra memory an algorithm needs relative to the input size. This includes memory used to store attendance status (present/absent) for each student and any auxiliary variables used during processing.

3. Growth of Operations with Input Size

- Each student ID is visited and processed exactly once, using a single pass (a single loop) through the list.
- If there are n students, the system performs exactly n operations (n comparisons/markings), one for each student ID.
- There is no nesting of loops and no repeated checking of the same ID, so the operation count grows linearly — doubling the number of students doubles the number of operations.

4. Growth of Memory Usage with Input Size

- The system needs to store one attendance record (e.g., present/absent flag) per student, so memory usage also grows linearly with n .
- Apart from this storage, only a constant number of auxiliary variables (like a loop counter) are used, which do not depend on n .

5. Asymptotic Complexity

Time Complexity: $O(n)$ — linear time, since each of the n IDs is checked exactly once.

Space Complexity: $O(n)$ — linear space, since one attendance record is stored per student.

6. Conclusion

Since the system performs a single, non-nested pass over the student list, both the running time and the memory required increase proportionally with the number of students. This makes the attendance-marking process an example of a linear-time, linear-space ($O(n)$) algorithm — efficient and scalable even for a large number of students.

Q72. Application: University Result Comparison System

Problem: A result analysis system compares each student's score with every other student's score to prepare rank statistics. Recall what pairwise comparison implies in complexity analysis, explain why execution time increases rapidly, and compute both the time complexity and space complexity using asymptotic notation.

1. Meaning of Pairwise Comparison

Pairwise comparison means that every element in a data set is compared with every other element at least once. In complexity analysis, this typically corresponds to a nested loop structure: for each of the n students, the system compares that student's score with the scores of the remaining $(n-1)$ students. This results in a quadratic number of comparisons rather than a linear one.

2. Why Execution Time Increases Rapidly

- For n students, comparing each student with every other student requires roughly $n \times n = n^2$ comparisons (using two nested loops, one inside the other).
- As n grows, the number of comparisons grows much faster than n itself — it grows with the square of n .
- Example: if there are 10 students, about 100 comparisons are needed; if there are 100 students, about 10,000 comparisons are needed; if there are 1,000 students, about 1,000,000 comparisons are needed.
- This shows that even a small increase in the number of students causes a very large increase in execution time, which is the hallmark of quadratic growth.

3. Time Complexity (Asymptotic Notation)

Since each of the n students is compared with the other $(n-1)$ students, the total number of comparisons is of the order $n(n-1)$, which simplifies to n^2 for large n .

```
Time Complexity =  $O(n^2)$  — quadratic time
```

4. Space Complexity (Asymptotic Notation)

The system needs to store the score of each of the n students, and typically also stores the computed rank or comparison result for each student. This requires memory proportional to n (a single array/list of scores and ranks), not n^2 , because the comparisons are usually done on the fly without storing every individual comparison result.

```
Space Complexity =  $O(n)$  — linear space
```

5. Conclusion

The result comparison system illustrates a classic quadratic-time algorithm: because every student's score is compared with every other student's score, the number of operations grows as $O(n^2)$, causing execution time to rise steeply as the number of students increases. However, since only the scores and final rank values need to be stored (not every comparison outcome), the space complexity remains linear at $O(n)$. This highlights an important distinction algorithm

analysis: a process can have low (linear) memory requirements while still having high (quadratic) time requirements.